LEKCJA 03

Lekcja 03: Kontrola wersji i GitHub

#lekcja #datascience #git #github #version-control #collaboration

LearnIT

learnit.com.pl

Spis treści

1 Czym jest kontrola wersji i dlaczego jest niezbędna w Data Science
2 Git - podstawowe komendy i workflow
3 Branching i Git Flow
4 GitHub - repozytoria, współpraca i Pull Requests
5 Opis problemu
6 Kroki do reprodukcji
7 Oczekiwane rozwiązanie
8 Środowisko
9 🌱 Aspekt środowiskowy
10 🤖 Narzędzia AI/ML wspierające pracę z Git i GitHub
11 📊 Przykładowy projekt Data Science z pełnym Git workflow
12 Opis
13 Struktura
14 Setup

15 🧪 Zadania do samodzielnej pracy

16 📚 Podsumowanie

17 📖 Dodatkowe Zasoby

W tej lekcji poznasz kluczowe narzędzie każdego Data Scientist - system kontroli wersji Git oraz platformę GitHub. Nauczysz się, jak śledzić zmiany w swoich projektach, współpracować z innymi oraz organizować pracę w sposób profesjonalny i efektywny. Zrozumiesz również, jak wersjonowanie kodu i danych może pomóc w oszczędnym wykorzystaniu zasobów i energii.

---

1. Czym jest kontrola wersji i dlaczego jest niezbędna w Data Science

> 📚 **Definicja**
>
> **Kontrola wersji** (version control) to system zarządzania zmianami w plikach projektu, który umożliwia śledzenie historii zmian, powrót do wcześniejszych wersji i współpracę z innymi osobami bez ryzyka utraty danych lub konfliktów.

Wprowadzenie

W Data Science pracujemy z wieloma elementami: kodem, danymi, notebookami Jupyter, modelami ML, raportami. Każdy z tych elementów ewoluuje w czasie - testujemy różne podejścia, poprawiamy błędy, eksperymentujemy z nowymi rozwiązaniami. Bez systemu kontroli wersji łatwo zgubić się w wersjach plików (`notebook_final.ipynb`, `notebook_final_v2.ipynb`, `notebook_final_NAPRAWDE_OSTATECZNY.ipynb`), stracić efekt pracy lub nie móc wrócić do działającego rozwiązania.

Git to najczęściej używany system kontroli wersji na świecie. Został stworzony przez Linusa Torvaldsa (twórcę Linuxa) i jest standardem w branży IT i Data Science.

Dlaczego Data Scientist potrzebuje Git?

1. **Historia zmian** - możesz zobaczyć, co zmieniłeś i kiedy
2. **Bezpieczeństwo** - możesz wrócić do poprzedniej wersji, jeśli coś pójdzie nie tak
3. **Współpraca** - wiele osób może pracować nad tym samym projektem równocześnie
4. **Eksperymentowanie** - możesz testować nowe pomysły bez strachu przed zepsuciem działającego kodu
5. **Reprodukowalność** - możesz odtworzyć dokładnie ten sam stan projektu z przeszłości
6. **Dokumentacja** - opisy commitów stanowią historię rozwoju projektu

Przykład 1: Problem bez kontroli wersji

Wyobraź sobie sytuację bez Git:

```
moj_projekt/
├── analiza.ipynb
├── analiza_kopia.ipynb
├── analiza_v2.ipynb
├── analiza_v2_poprawki.ipynb
├── analiza_final.ipynb
├── analiza_final_NAPRAWDE.ipynb
└── analiza_final_po_rozmowie_z_szefem.ipynb
```

Problemy:

- Która wersja jest właściwa?
- Co się zmieniło między wersjami?
- Jak wrócić do wersji sprzed tygodnia?
- Jak współpracować z kolegą z zespołu?

Przykład 2: Ten sam projekt z Git

```
# Zobaczenie historii zmian
git log --oneline

# Wynik:
# a3f21c8 Dodanie analizy korelacji
# 9b4d3e1 Poprawka obsługi brakujących danych
```

```
# 7c2a8f5 Pierwsza wersja eksploracji danych
# 4e1b9d2 Inicjalizacja projektu
```

Korzyści:

- Jeden plik `analiza.ipynb`
- Pełna historia zmian z opisami
- Możliwość powrotu do dowolnej wersji
- Czytelna dokumentacja rozwoju projektu

Przykład 3: Workflow Data Scientist z Git

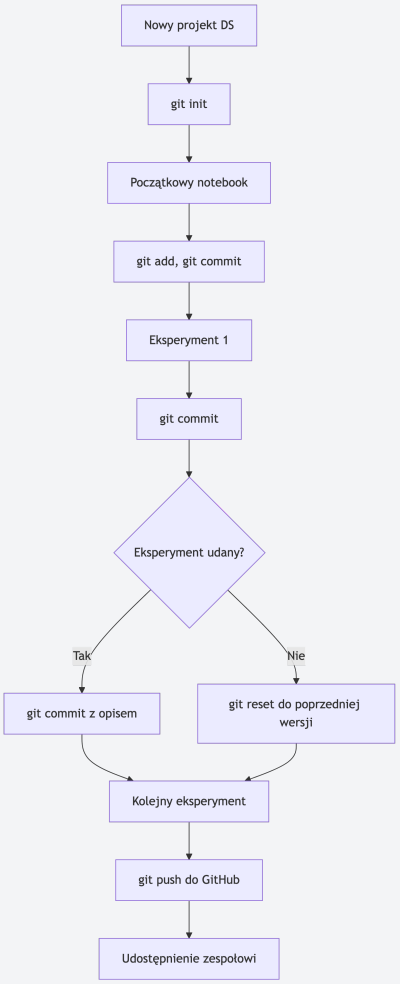


Diagram 1

> 💡 **Wskazówka**
>
> **Dobra praktyka DS:** Commituj kod po każdym znaczącym kroku analizy (np. "zakończono czyszczenie danych", "dodano model baseline", "poprawiono feature engineering"). Dzięki temu masz punkty kontrolne, do których możesz wrócić.

> ⚠️ **Uwaga**
>
> **Uwaga:** NIE commituj dużych plików danych (powyżej 50-100 MB) bezpośrednio do Git. Zamiast tego używaj Git LFS (Large File Storage) lub przechowuj dane w dedykowanych miejscach (cloud storage, bazy danych), a w repozytorium trzymaj tylko skrypty do ich pobrania.

> 📝 **Notatka**
>
> W Data Science często wersjonujemy: kod Pythona (.py), notebooki Jupyter (.ipynb), skrypty do preprocessingu danych, konfiguracje modeli, dokumentację. Natomiast same duże datasety i wytrenowane modele przechowujemy osobno.

---

2. Git - podstawowe komendy i workflow

> 📚 **Definicja**
>
> **Git** to rozproszony system kontroli wersji, który śledzi zmiany w plikach poprzez **commity** - migawki (snapshots) stanu projektu w danym momencie czasu. Każdy commit ma unikalny identyfikator, autora, czas i opis zmian.

Wprowadzenie

Git działa na zasadzie robienia "zdjęć" (commitów) stanu Twojego projektu. Każde takie "zdjęcie" zawiera informację o tym, jakie pliki się zmieniły, kto i kiedy je zmienił oraz krótki opis zmian. Możesz przeglądać tę historię, porównywać zmiany i wracać do dowolnego momentu w przeszłości.

Podstawowa struktura Git



Diagram 2

Trzy główne obszary:

1. **Working Directory** - Twój katalog roboczy z plikami
2. **Staging Area** - miejsce przejściowe, gdzie przygotowujesz zmiany do commita
3. **Repository** - lokalna baza danych z historią commitów

Przykład 1: Inicjalizacja projektu Data Science

```
# Tworzenie nowego folderu projektu
mkdir credit_risk_analysis
cd credit_risk_analysis

# Inicjalizacja repozytorium Git
git init

# Wynik:
# Initialized empty Git repository in /Users/username/credit_risk_analysis/.git/

# Sprawdzenie statusu
git status

# Wynik:
# On branch master
# No commits yet
# nothing to commit (create/copy files and use "git add" to track)
```

Co się stało?

- Utworzono ukryty folder `.git/`, który przechowuje całą historię projektu
- Jesteś na domyślnej gałęzi (branch) o nazwie `master` lub `main`

- Repozytorium jest puste, gotowe do dodania plików

Przykład 2: Pierwszy commit

```
# Tworzenie struktury projektu DS
mkdir data notebooks models

# Tworzenie pliku README
echo "# Credit Risk Analysis Project" > README.md

# Tworzenie prostego notatnika (symulacja)
echo "# Exploratory Data Analysis" > notebooks/eda.ipynb

# Sprawdzenie statusu
git status

# Wynik:
# On branch master
# Untracked files:
#   README.md
#   notebooks/

# Dodanie plików do staging area
git add README.md
git add notebooks/eda.ipynb

# Alternatywnie - dodanie wszystkiego:
# git add .

# Sprawdzenie co jest w staging
git status

# Wynik:
# Changes to be committed:
#   new file:   README.md
#   new file:   notebooks/eda.ipynb

# Utworzenie commita z opisem
git commit -m "Inicjalizacja projektu: struktura folderów i README"

# Wynik:
# [master (root-commit) a1b2c3d] Inicjalizacja projektu: struktura folderów i README
# 2 files changed, 2 insertions(+)
```

```
# create mode 100644 README.md
# create mode 100644 notebooks/eda.ipynb
```

Wyjaśnienie kroków:

1. `git add` - dodaje pliki do staging area (przygotowanie do commita)
2. `git commit -m "opis"` - tworzy commit z opisem zmian
3. Commit otrzymuje unikalny identyfikator (hash), np. `a1b2c3d`

Przykład 3: Dalsza praca i historia zmian

```
# Edycja pliku (np. dodanie sekcji do README)
echo "## Dataset: German Credit Data" >> README.md

# Dodanie nowego pliku
echo "pandas==2.0.0\nscikit-learn==1.3.0" > requirements.txt

# Sprawdzenie zmian
git status

# Wynik:
# Changes not staged for commit:
#   modified:   README.md
# Untracked files:
#   requirements.txt

# Zobaczenie różnic w zmienionych plikach
git diff README.md

# Wynik pokaże dodaną linię:
# +## Dataset: German Credit Data

# Dodanie i commit
git add README.md requirements.txt
git commit -m "Dodanie opisu datasetu i requirements.txt"

# Zobaczenie historii
git log

# Wynik:
# commit b4c5d6e (HEAD -> master)
# Author: Jan Kowalski <jan@example.com>
# Date:   Thu Nov 21 10:30:00 2025 +0100
```

```
#
#     Dodanie opisu datasetu i requirements.txt
#
# commit a1b2c3d
# Author: Jan Kowalski <jan@example.com>
# Date:   Thu Nov 21 10:00:00 2025 +0100
#
#     Inicjalizacja projektu: struktura folderów i README

# Bardziej zwięzła historia
git log --oneline

# Wynik:
# b4c5d6e Dodanie opisu datasetu i requirements.txt
# a1b2c3d Inicjalizacja projektu: struktura folderów i README
```

Schemat działania git add i git commit

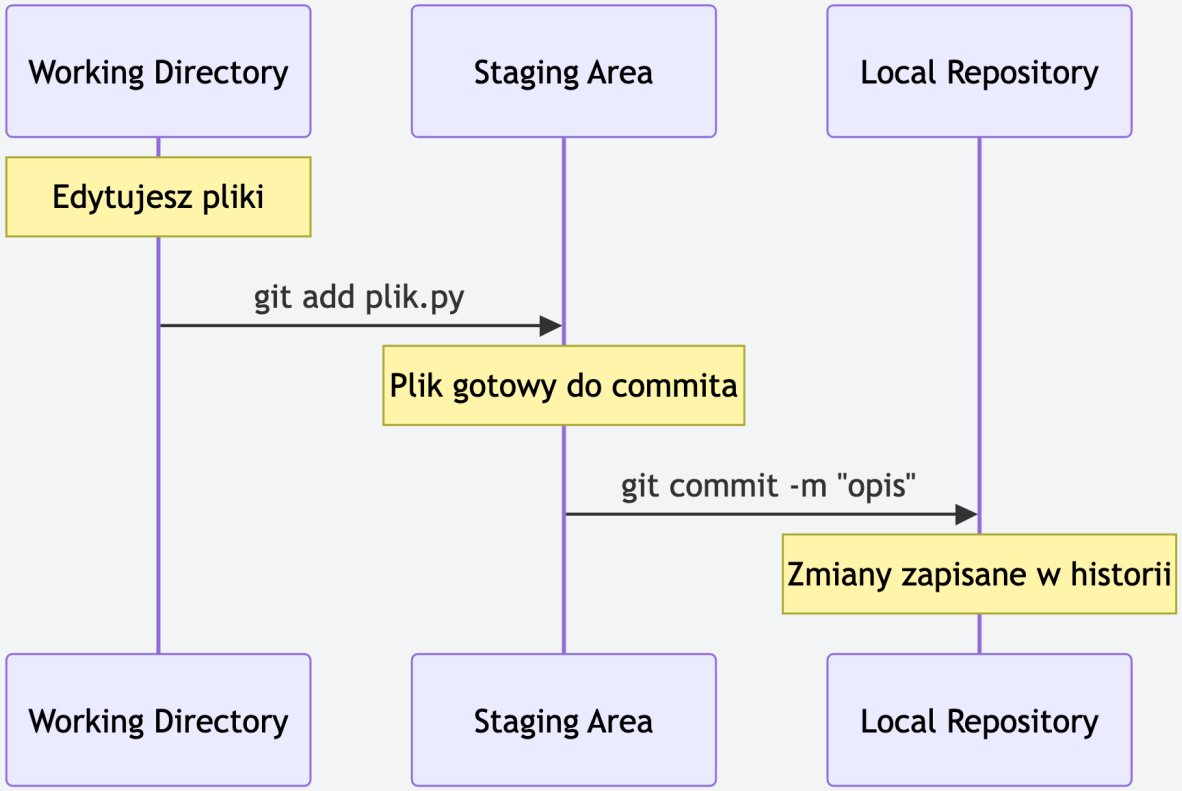


Diagram 3

💡 **Wskazówka**

Dobre praktyki commitowania: - Commit często (po każdej logicznej zmianie) - Pisz zrozumiałe opisy commitów (co i dlaczego) - Nie commituj połowy pracy - każdy commit powinien być "działający" - W Data Science: commituj po zakończeniu etapu (EDA, preprocessing, modeling, evaluation)

⚠️ **Uwaga**

Częste błędy początkujących: - Zapomniane `git add` przed `git commit` - commit będzie pusty - Zbyt ogólne opisy: "poprawki" zamiast "Poprawiono obsługę wartości NaN w kolumnie income" - Commitowanie wrażliwych danych (hasła, klucze API) - NIGDY tego nie rób!

✅ **Sukces**

Best practice w DS: Twórz `.gitignore` na początku projektu, aby wykluczyć: duże pliki danych, modele `.pkl`, pliki środowiska (`.env`), cache Pythona (`__pycache__/` , `*.pyc`), foldery środowisk wirtualnych (`venv/` , `env/`).

KURS DATA SCIENTIST ONLINE + AI

Chcesz zostać Data Scientist?

Ten materiał to fragment kursu Data Scientist Online + AI. Dołącz do nas i zdobądź praktyczne umiejętności pod okiem ekspertów z branży.

✓ Zajęcia na żywo 2x/tydzień ✓ Projekty praktyczne ✓ Certyfikat ukończenia

Zobacz szczegóły kursu →

3. Branching i Git Flow

> 📚 **Definicja**
>
> **Branch** (gałąź) to niezależna linia rozwoju projektu. Pozwala pracować nad nową funkcjonalnością lub eksperymentem bez wpływu na główną (stabilną) wersję kodu. **Git Flow** to strategia organizacji pracy z branchami, która definiuje kiedy i jak tworzyć różne typy gałęzi.

Wprowadzenie

Branching to jedna z najpotężniejszych funkcji Git. W Data Science pozwala to na:

- Testowanie nowych pomysłów bez ryzyka zepsucia działającego modelu
- Równoległą pracę nad różnymi eksperymentami
- Oddzielenie kodu produkcyjnego od eksperymentalnego
- Współpracę wielu osób bez konfliktów

Jak działają branchi?

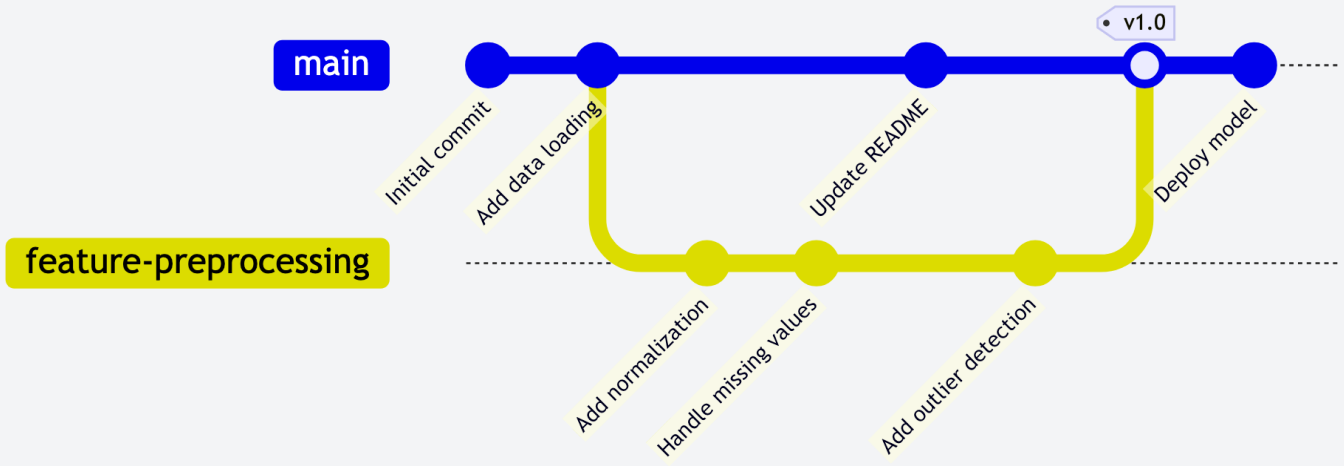


Diagram 4

Wyjaśnienie:

- `main` (lub `master`) - główna gałąź, zawiera stabilny kod
- `feature-preprocessing` - oddzielna gałąź do pracy nad preprocessingiem
- Po zakończeniu pracy, gałąź można połączyć (merge) z `main`

Przykład 1: Tworzenie i przełączanie między branchami

```
# Sprawdzenie obecnego brancha
git branch

# Wynik:
# * master  (gwiazdka oznacza aktywny branch)

# Utworzenie nowego brancha dla eksperymentu
git branch experiment-xgboost

# Sprawdzenie wszystkich branchy
git branch

# Wynik:
# * master
#   experiment-xgboost

# Przełączenie się na nowy branch
git checkout experiment-xgboost

# Wynik:
# Switched to branch 'experiment-xgboost'

# Lub krócej - utworzenie i przełączenie w jednej komendzie:
git checkout -b experiment-random-forest

# Teraz jesteś na branchu experiment-random-forest
# Wszystkie commity będą zapisywane tutaj, nie wpływając na master
```

Przykład 2: Praca na osobnym branchu

```
# Jesteś na branchu experiment-xgboost
git branch
# * experiment-xgboost
#   master
```

```
# Dodajemy nowy notebook z eksperymentem
echo "# XGBoost Experiment" > notebooks/xgboost_model.ipynb

# Dodajemy kod do pliku
echo "import xgboost as xgb" >> notebooks/xgboost_model.ipynb

# Commitujemy na tym branchu
git add notebooks/xgboost_model.ipynb
git commit -m "Dodano eksperyment z XGBoost"

# Przełączamy się z powrotem na master
git checkout master

# Sprawdzamy listę plików - xgboost_model.ipynb nie ma!
ls notebooks/
# Wynik: eda.ipynb (tylko pliki z mastera)

# Przełączamy z powrotem na experiment-xgboost
git checkout experiment-xgboost

# Sprawdzamy listę plików - xgboost_model.ipynb jest!
ls notebooks/
# Wynik: eda.ipynb, xgboost_model.ipynb
```

Co się stało?

- Na branchu `experiment-xgboost` dodałeś nowy plik
- Branch `master` pozostał niezmieniony
- Możesz swobodnie przełączać się między branchami
- Każdy branch ma swoją własną historię commitów

Przykład 3: Merge - łączenie branchy

```
# Załóżmy, że eksperyment się udał i chcesz dodać go do mastera

# Przełącz się na master
git checkout master

# Połącz branch experiment-xgboost z masterem
git merge experiment-xgboost
```

```
# Wynik:
# Updating a1b2c3d..e5f6g7h
# Fast-forward
#  notebooks/xgboost_model.ipynb | 2 ++
#  1 file changed, 2 insertions(+)
#  create mode 100644 notebooks/xgboost_model.ipynb

# Teraz master zawiera zmiany z experiment-xgboost
ls notebooks/
# Wynik: eda.ipynb, xgboost_model.ipynb

# Możesz usunąć już niepotrzebny branch
git branch -d experiment-xgboost

# Wynik:
# Deleted branch experiment-xgboost (was e5f6g7h)
```

Git Flow w praktyce

Git Flow to strategia zarządzania branchami. Definiuje kilka typów gałęzi:

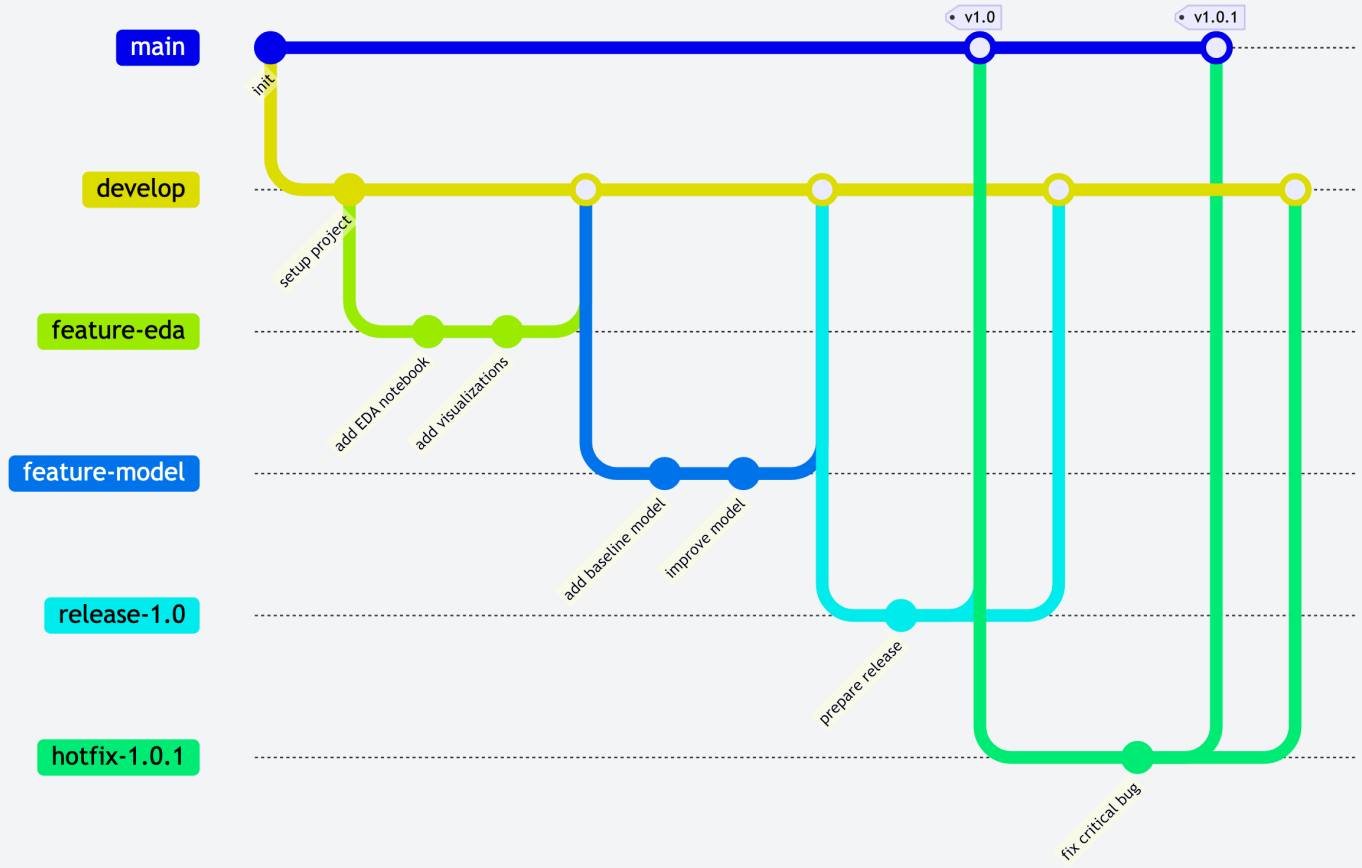


Diagram 5

Typy branchy w Git Flow:

1. **main** (master) - produkcja, tylko stabilny kod
2. **develop** - branch deweloperski, integracja funkcji
3. **feature/** - nowe funkcjonalności (np. `feature-new-algorithm`)
4. **release/** - przygotowanie do wydania (np. `release-1.0`)
5. **hotfix/** - pilne poprawki na produkcji (np. `hotfix-bug-in-preprocessing`)

Przykład zastosowania Git Flow w projekcie DS

```
# Inicjalizacja projektu
git checkout -b develop  # utworzenie brancha develop

# Feature 1: Eksploracja danych
git checkout -b feature-eda
```

```
# ... praca nad EDA ...
git add .
git commit -m "Zakończono eksploracyjną analizę danych"
git checkout develop
git merge feature-eda

# Feature 2: Model baseline
git checkout -b feature-baseline-model
# ... implementacja modelu baseline ...
git add .
git commit -m "Dodano model baseline (Logistic Regression)"
git checkout develop
git merge feature-baseline-model

# Feature 3: Advanced model
git checkout -b feature-xgboost
# ... implementacja XGBoost ...
git add .
git commit -m "Dodano XGBoost model z hyperparameter tuning"
git checkout develop
git merge feature-xgboost

# Przygotowanie release
git checkout -b release-1.0
# ... testy, dokumentacja, drobne poprawki ...
git add .
git commit -m "Przygotowanie release 1.0"

# Merge do main (produkcja)
git checkout main
git merge release-1.0
git tag -a v1.0 -m "Pierwsza wersja produkcyjna modelu credit risk"

# Merge z powrotem do develop
git checkout develop
git merge release-1.0
```

💡 Wskazówka

W Data Science często stosujemy uproszczony Git Flow: - `main` - kod produkcyjny, wytrenowane modele - `develop` - aktywny development - `experiment-*` - branchi eksperymentalne dla testowania różnych modeli - Każdy członek zespołu pracuje na swoim branchu, potem merge do develop

📝 Notatka

Konflikty podczas merge: Gdy dwie osoby zmieniają ten sam fragment kodu, Git nie wie, którą wersję wybrać. Trzeba wtedy ręcznie rozwiązać konflikt, edytując plik i wybierając właściwe zmiany. Git oznaczy konflikty w pliku znakami `<<<<<<<` , `=======` , `>>>>>>>` .

⚠️ Uwaga

Notebooki Jupyter (`.ipynb`) są plikami JSON, co powoduje, że konflikty są trudne do rozwiązania. **Rozwiązanie:** używaj narzędzi jak `nbdime` do diffowania i mergowania notebooków lub konwertuj notebooki do `.py` przed commitowaniem.

---

4. GitHub - repozytoria, współpraca i Pull Requests

📚 Definicja

GitHub to platforma hostingowa dla repozytoriów Git, która dodaje funkcje współpracy: Pull Requests, Issues, Projekty, GitHub Actions (CI/CD), a także umożliwia dzielenie się kodem open-source i budowanie portfolio.

Wprowadzenie

GitHub jest jak "social media" dla kodu. Tutaj programiści i Data Scientists:

- Przechowują swoje projekty w chmurze
- Współpracują z innymi
- Dzielą się kodem open-source
- Budują portfolio projektów
- Automatyzują testy i deployment (CI/CD)

Alternatywy: GitLab, Bitbucket, ale GitHub jest najpopularniejszy.

Architektura pracy z GitHub

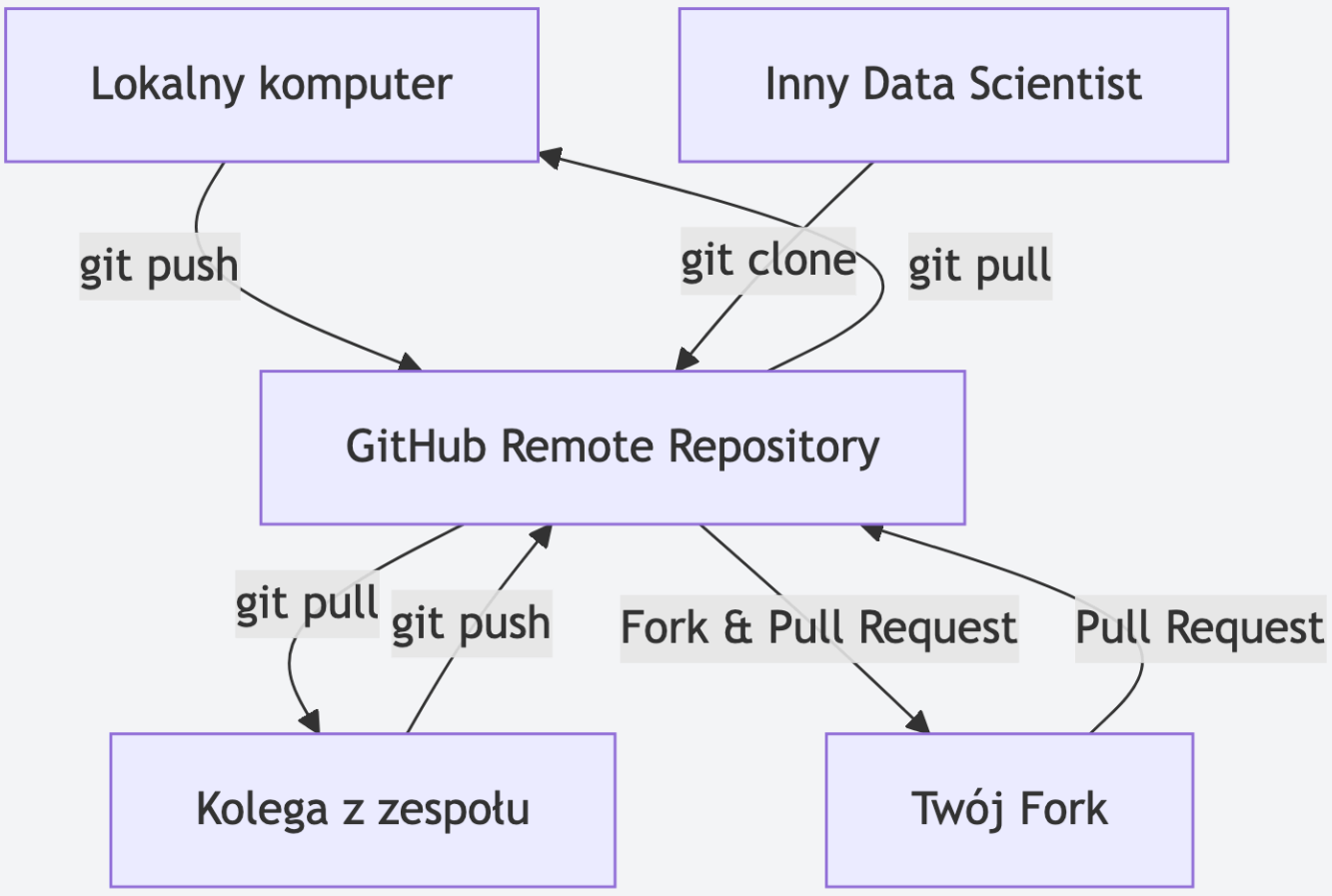


Diagram 6

Przykład 1: Utworzenie repozytorium na GitHub i push

```
# Krok 1: Utwórz repozytorium na GitHub (przez przeglądarkę)
# - Zaloguj się na github.com
# - Kliknij "New repository"
# - Nazwa: credit-risk-analysis
# - Opis: Machine Learning model for credit risk assessment
# - Public lub Private
# - Nie zaznaczaj "Initialize with README" (mamy już lokalne repo)
# - Create repository

# Krok 2: Połącz lokalne repo z GitHub
git remote add origin https://github.com/username/credit-risk-analysis.git

# Sprawdzenie remote
git remote -v

# Wynik:
# origin  https://github.com/username/credit-risk-analysis.git (fetch)
# origin  https://github.com/username/credit-risk-analysis.git (push)

# Krok 3: Wypchnij kod na GitHub
git push -u origin master

# Wynik:
# Enumerating objects: 10, done.
# Counting objects: 100% (10/10), done.
# Delta compression using up to 8 threads
# Compressing objects: 100% (7/7), done.
# Writing objects: 100% (10/10), 1.2 KiB | 1.2 MiB/s, done.
# Total 10 (delta 1), reused 0 (delta 0)
# To https://github.com/username/credit-risk-analysis.git
#  * [new branch]      master -> master

# Teraz Twój kod jest na GitHub!
```

Co się stało?

- `git remote add origin URL` - dodałeś zdalny repository o nazwie `origin`
- `git push -u origin master` - wypchnąłeś branch `master` na GitHub
- `-u` ustawia `origin master` jako domyślny upstream (następnym razem wystarczy `git push`)

Przykład 2: Klonowanie i praca zespołowa

```
# Kolega z zespołu chce dołączyć do projektu

# Klonowanie repozytorium
git clone https://github.com/username/credit-risk-analysis.git

# Wynik:
# Cloning into 'credit-risk-analysis'...
# remote: Enumerating objects: 10, done.
# remote: Counting objects: 100% (10/10), done.
# remote: Compressing objects: 100% (7/7), done.
# remote: Total 10 (delta 1), reused 10 (delta 1)
# Unpacking objects: 100% (10/10), done.

cd credit-risk-analysis

# Sprawdzenie remote - jest już skonfigurowany
git remote -v
# origin  https://github.com/username/credit-risk-analysis.git (fetch)
# origin  https://github.com/username/credit-risk-analysis.git (push)

# Kolega tworzy nowy branch dla swojej pracy
git checkout -b feature-data-visualization

# Dodaje kod
echo "# Data Visualization" > notebooks/visualizations.ipynb

git add notebooks/visualizations.ipynb
git commit -m "Dodano notebook z wizualizacjami danych"

# Wypycha swój branch na GitHub
git push -u origin feature-data-visualization
```

Przykład 3: Pull Request workflow

Pull Request (PR) to prośba o zaakceptowanie zmian z jednego brancha do drugiego. To główny mechanizm współpracy na GitHub.

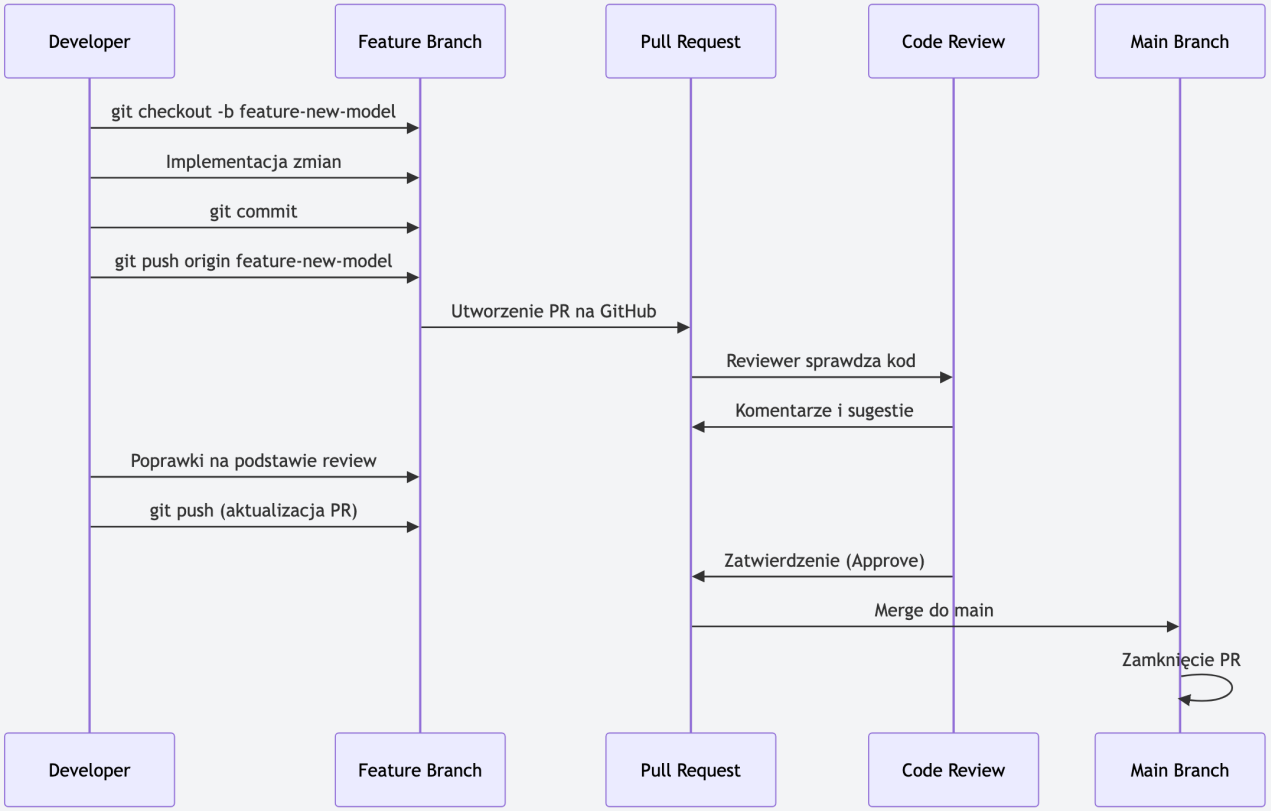


Diagram 7

Kroki tworzenia Pull Request:

1. **Push brancha na GitHub:**

```
git push origin feature-data-visualization
```

1. **Utworzenie PR przez GUI GitHub:**

- Wejdź na stronę repo na GitHub
- Zobaczysz banner: "feature-data-visualization had recent pushes - Compare & pull request"
- Kliknij "Compare & pull request"
- Wypełnij tytuł: "Dodanie wizualizacji danych"
- Wypełnij opis:

```
## Opis zmian
  - Dodano notebook z wizualizacjami rozkładów zmiennych
```

```
- Utworzono wykresy korelacji
- Dodano analizę outlierów

## Checklist
- [x] Kod działa lokalnie
- [x] Dodano komentarze
- [ ] Wymaga review od @team-lead
```

- Wybierz reviewers (osoby do sprawdzenia kodu)
- Kliknij "Create pull request"

1. **Code Review:**

- Reviewer sprawdza kod, dodaje komentarze
- Developer wprowadza poprawki i pushuje
- PR automatycznie się aktualizuje

1. **Merge:**

- Po zatwierdzeniu, kliknij "Merge pull request"
- Wybierz typ merge (zwykle "Squash and merge" lub "Create a merge commit")
- Zatwierdź merge

GitHub Issues - zarządzanie zadaniami

Issues to system zgłoszeń zadań, bugów, pomysłów:

```
# Przykład Issue:

Tytuł: Model overfittuje na danych treningowych
Labels: bug, model-improvement
Assignee: @data-scientist-anna

## Opis problemu
Model Random Forest osiąga 98% accuracy na zbiorze treningowym,
ale tylko 75% na walidacyjnym.

## Kroki do reprodukcji
1. Uruchom notebook `notebooks/random_forest_model.ipynb`
2. Sprawdź metryki na komórce #15
```

```
## Oczekiwane rozwiązanie
- Dodać regularyzację (max_depth, min_samples_split)
- Zastosować cross-validation
- Porównać z modelem baseline

## Środowisko
- Python 3.10
- scikit-learn 1.3.0
```

> 💡 **Wskazówka**
>
> **Dobre praktyki PR w Data Science:** - Jeden PR = jedna funkcjonalność (nie mieszaj EDA + modeling + deployment) - Opisuj co i dlaczego zmieniłeś - Dodaj screenshoty wykresów jeśli są istotne - Linkuj do related Issues (#123) - Poproś o review konkretne osoby (@username)

> ✅ **Sukces**
>
> **GitHub Projects** to tablica Kanban do zarządzania zadaniami DS: - Kolumny: To Do, In Progress, Review, Done - Karty to Issues lub PR - Wizualizacja postępu projektu ML

> 📝 **Notatka**
>
> **GitHub Actions** pozwala automatyzować: - Uruchamianie testów przy każdym PR - Automatyczne trenowanie modeli - Deployment do produkcji - Generowanie raportów z metrykami modeli

---

KURS DATA SCIENTIST ONLINE + AI

Chcesz zostać Data Scientist?

Ten materiał to fragment kursu Data Scientist Online + AI. Dołącz do nas i zdobądź praktyczne umiejętności pod okiem ekspertów z branży.

✓ Zajęcia na żywo 2x/tydzień ✓ Projekty praktyczne ✓ Certyfikat ukończenia

Zobacz szczegóły kursu →

5. 🌱 Aspekt środowiskowy

Kontrola wersji i GitHub mogą znacząco wpłynąć na redukcję zużycia energii i zasobów w projektach Data Science:

1. Wersjonowanie zamiast powielania plików

Problem bez Git:

```
projekt/
├── model_v1.pkl (500 MB)
├── model_v2.pkl (500 MB)
├── model_v3.pkl (500 MB)
├── model_final.pkl (500 MB)
└── model_final_OSTATECZNY.pkl (500 MB)
Total: 2.5 GB lokalnie + backup + chmura = ~10 GB
```

Rozwiązanie z Git:

```
projekt/
├── .git/ (historia zmian w kodzie, nie w dużych plikach)
```

```
├── model.pkl (500 MB, nie w repo)
└── train_model.py (tylko kod wersjonowany)
Total wersjonowany: ~50 KB
```

Korzyści środowiskowe:

- Mniej miejsca na dysku = mniej energii potrzebnej do przechowywania
- Mniej transferu danych przez sieć
- Mniej backupów = mniej zużycia energii serwerów

2. Repozytoria chmurowe zamiast lokalnych kopii

Zamiast: Każdy w zespole ma pełną kopię projektu + backup lokalny **Lepiej:** Jedno centralne repozytorium na GitHub, team clonuje tylko potrzebne

```
# Przykład: klonowanie tylko najnowszej wersji (shallow clone)
# zamiast całej historii

# Standardowe klonowanie - cała historia (może być GB danych)
git clone https://github.com/username/large-ml-project.git

# Shallow clone - tylko ostatni commit (oszczędność energii i czasu)
git clone --depth 1 https://github.com/username/large-ml-project.git

# Oszczędność:
# Pełne repo: 500 MB (cała historia)
# Shallow: 50 MB (tylko obecny stan)
# = 90% mniej danych do pobrania
```

3. Gitignore - wykluczanie zbędnych plików

Unikaj commitowania plików, które:

- Generują się automatycznie
- Są bardzo duże
- Można odtworzyć z kodu

```
# .gitignore dla projektu Data Science

# Duże pliki danych - nie commituj!
```

```
data/raw/*.csv
data/raw/*.parquet
*.h5
*.hdf5

# Wytrenowane modele - przechowuj osobno (MLflow, cloud)
models/*.pkl
models/*.h5
models/*.pt
*.ckpt

# Cache Pythona
__pycache__/
*.pyc
*.pyo

# Jupyter checkpoints
.ipynb_checkpoints/

# Środowiska wirtualne
venv/
env/
.conda/

# Pliki systemowe
.DS_Store
Thumbs.db

# Logi
logs/
*.log

# Tymczasowe pliki
tmp/
temp/
```

Korzyść: Repo ma 10 MB zamiast 10 GB - transfer i storage o 99.9% mniejszy.

4. Efektywne współdzielenie kodu

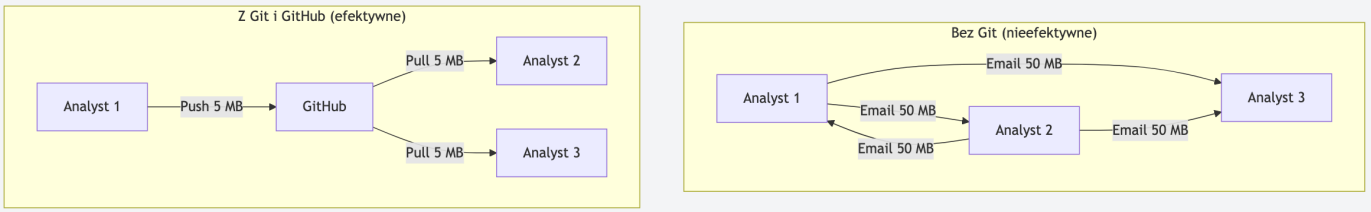


Diagram 8

Obliczenia:

- Bez Git: 6 transferów × 50 MB = 300 MB przesłanych danych
- Z Git: 1 push (5 MB) + 2 pull (5 MB każdy) = 15 MB
- **Oszczędność: 95%** ruchu sieciowego i energii potrzebnej do transferu

5. Przykład monitoringu zużycia zasobów w Git workflow

```
# Skrypt do oszacowania oszczędności dzięki Git

import os

def calculate_storage_savings():
    """
    Porównanie: wielokrotne wersje plików vs Git
    """
    # Przykładowy projekt DS bez Git
    notebook_size_mb = 5
    model_size_mb = 500
    data_size_mb = 1000
    versions = 10

    without_git_mb = (notebook_size_mb + model_size_mb + data_size_mb) * versions

    # Z Git: tylko kod wersjonowany, modele i dane osobno
    with_git_mb = notebook_size_mb * 0.1  # historia kodu (kompresja Git)
    with_git_mb += model_size_mb  # najnowszy model (nie w Git)
    with_git_mb += data_size_mb  # dane (nie w Git)
```

```
    savings_mb = without_git_mb - with_git_mb
    savings_percent = (savings_mb / without_git_mb) * 100

    # Szacunkowe zużycie energii (uproszczone)
    # 1 GB storage ≈ 0.002 kWh/rok
    energy_without_kwh = (without_git_mb / 1000) * 0.002
    energy_with_kwh = (with_git_mb / 1000) * 0.002
    energy_saved_kwh = energy_without_kwh - energy_with_kwh

    print(f"📊 Analiza oszczędności Git vs wielokrotne kopie")
    print(f"Bez Git: {without_git_mb:,.0f} MB ({without_git_mb/1000:.1f} GB)")
    print(f"Z Git: {with_git_mb:,.0f} MB ({with_git_mb/1000:.1f} GB)")
    print(f"💾 Oszczędność miejsca: {savings_mb:,.0f} MB ({savings_percent:.1f}%)")
    print(f"⚡ Oszczędność energii: {energy_saved_kwh:.4f} kWh/rok")
    print(f"🌍 To odpowiednik {energy_saved_kwh * 0.5:.2f} kg CO₂/rok")

calculate_storage_savings()

# Wynik:
# 📊 Analiza oszczędności Git vs wielokrotne kopie
# Bez Git: 15,050 MB (15.0 GB)
# Z Git: 1,500 MB (1.5 GB)
# 💾 Oszczędność miejsca: 13,550 MB (90.0%)
# ⚡ Oszczędność energii: 0.0271 kWh/rok
# 🌍 To odpowiednik 0.01 kg CO₂/rok
```

6. Best Practices Green IT dla Git/GitHub

✅ **DO:**

- Używaj `.gitignore` do wykluczenia dużych plików
- Commituj tylko kod i konfiguracje
- Przechowuj dane i modele w dedykowanych miejscach (S3, GCS, DVC)
- Korzystaj z shallow clone dla dużych repozytoriów
- Usuwaj stare branche po merge
- Archiwizuj nieaktywne repozytoria

❌ **DON'T:**

- Nie commituj datasetów > 50 MB
- Nie commituj wytrenowanych modeli > 100 MB

- Nie duplikuj całego repo dla eksperymentów (użyj branchy)
- Nie pushuj każdej małej zmiany (commituj lokalnie, pushuj raz dziennie)
- Nie trzymaj wielu forków tego samego repo

```
# Narzędzie DVC (Data Version Control) - wersjonowanie danych
# Alternatywa dla Git do dużych plików

# Instalacja
!pip install dvc dvc-s3

# Inicjalizacja
!dvc init

# Dodanie dużego pliku do DVC zamiast Git
!dvc add data/large_dataset.csv

# DVC utworzy plik data/large_dataset.csv.dvc (mały, można commitować)
# Rzeczywiste dane idą do storage (S3)

!git add data/large_dataset.csv.dvc
!git commit -m "Dodano dataset (DVC tracked)"

# Pobieranie danych
!dvc pull  # ściąga dane z S3

# Korzyść: Git jest lekki, dane w efektywnym storage
```

---

6. 🤖 Narzędzia AI/ML wspierające pracę z Git i GitHub

1. GitHub Copilot

AI assistant, który:

- Podpowiada komendy Git w terminalu
- Pomaga pisać commit messages
- Sugeruje rozwiązania konfliktów

2. ChatGPT / Claude dla Git

Możesz zapytać:

```
Prompt: "Napisz komendę Git, która utworzy nowy branch, przełączy się na niego i wypchnie na GitHub"

Odpowiedź:
git checkout -b feature-new-experiment && git push -u origin feature-new-experiment
```

3. Automatyczne generowanie .gitignore

```python
# Python snippet do generowania .gitignore dla projektu DS

import requests

def generate_gitignore(project_type="python"):
    """
    Pobiera szablon .gitignore z gitignore.io
    """
    url = f"https://www.toptal.com/developers/gitignore/api/{project_type},jupyternotebooks,visualstudiocode"

    response = requests.get(url)

    if response.status_code == 200:
        with open(".gitignore", "w") as f:
            f.write(response.text)
        print("✅ .gitignore created!")
        return response.text
    else:
        print("❌ Failed to generate .gitignore")
        return None

# Użycie
generate_gitignore("python")
```

4. Automatyczne commit messages z AI

```
# Użycie narzędzia aicommits (npm package)
# Generuje commit message na podstawie zmian w kodzie

# Instalacja (wymaga Node.js)
!npm install -g aicommits

# Konfiguracja z kluczem OpenAI
!aicommits config set OPENAI_KEY=your-api-key

# Użycie - zamiast git commit -m "..."
!aicommits

# AI przeanalizuje zmiany i zaproponuje message:
# "feat: add XGBoost model with hyperparameter tuning and cross-validation"
```

5. GitHub Actions dla MLOps

Automatyzacja:

```
# .github/workflows/train-model.yml

name: Train Model on Push

on:
  push:
    branches: [ develop ]

jobs:
  train:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.10'

      - name: Install dependencies
        run: |
```

```
        pip install -r requirements.txt

    - name: Train model
      run: |
        python src/train_model.py

    - name: Evaluate model
      run: |
        python src/evaluate_model.py

    - name: Upload metrics
      run: |
        python src/log_metrics.py
```

Korzyść: Każdy push automatycznie trenuje model i raportuje wyniki.

KURS DATA SCIENTIST ONLINE + AI

Chcesz zostać Data Scientist?

Ten materiał to fragment kursu Data Scientist Online + AI. Dołącz do nas i zdobądź praktyczne umiejętności pod okiem ekspertów z branży.

✓ Zajęcia na żywo 2x/tydzień ✓ Projekty praktyczne ✓ Certyfikat ukończenia

Zobacz szczegóły kursu →

7. 📊 Przykładowy projekt Data Science z pełnym Git workflow

Zobaczmy kompletny przykład projektu od zera do Pull Request:

```
# === INICJALIZACJA PROJEKTU ===
```

```
# 1. Utworzenie folderu i inicjalizacja Git
mkdir fraud-detection-ml
cd fraud-detection-ml
git init

# 2. Utworzenie struktury
mkdir -p data/{raw,processed} notebooks src models

# 3. Utworzenie .gitignore
cat > .gitignore << EOF
# Data
data/raw/*.csv
data/processed/*.pkl

# Models
models/*.pkl
models/*.h5

# Python
__pycache__/
*.pyc
.ipynb_checkpoints/

# Environment
venv/
.env
EOF

# 4. Utworzenie README
cat > README.md << EOF
# Fraud Detection ML Project

## Opis
Model ML do wykrywania fraudulentnych transakcji.

## Struktura
- data/ - datasets (nie commitowane)
- notebooks/ - Jupyter notebooks
- src/ - kod źródłowy
- models/ - wytrenowane modele (nie commitowane)

## Setup
\`\`\`bash
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

```
\`\`\`
EOF

# 5. requirements.txt
cat > requirements.txt << EOF
pandas==2.0.0
scikit-learn==1.3.0
matplotlib==3.7.0
seaborn==0.12.0
jupyter==1.0.0
EOF

# 6. Pierwszy commit
git add .
git commit -m "Inicjalizacja projektu: struktura i konfiguracja"

# === PRACA NAD EDA ===

# 7. Utworzenie brancha dla EDA
git checkout -b feature-eda

# 8. Utworzenie notatnika
cat > notebooks/01_eda.ipynb << EOF
{
 "cells": [
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": ["# Eksploracyjna Analiza Danych"]
  },
  {
   "cell_type": "code",
   "metadata": {},
   "source": ["import pandas as pd\nimport matplotlib.pyplot as plt"]
  }
 ]
}
EOF

git add notebooks/01_eda.ipynb
git commit -m "Dodano notebook EDA z podstawowymi importami"

# Kontynuacja pracy...
# (symulacja edycji notatnika)
git add notebooks/01_eda.ipynb
git commit -m "Dodano analizę rozkładów zmiennych"
```

```
git add notebooks/01_eda.ipynb
git commit -m "Dodano macierz korelacji i identyfikację outlierów"

# 9. Push do GitHub
git push -u origin feature-eda

# === PULL REQUEST ===

# 10. Utworzenie PR przez GitHub GUI
# Title: "Eksploracyjna analiza danych"
# Description:
# - Analiza rozkładów
# - Korelacje między zmiennymi
# - Identyfikacja outlierów
# - Wizualizacje

# === REVIEW I MERGE ===

# 11. Po zatwierdzeniu PR, merge do main
git checkout main
git pull origin main  # pobierz zmiany po merge

# === KOLEJNA FUNKCJONALNOŚĆ ===

# 12. Nowy branch dla modelu
git checkout -b feature-baseline-model

# ... praca nad modelem ...

git add src/train_model.py
git commit -m "Dodano baseline model (Logistic Regression)"

git push -u origin feature-baseline-model

# ... Pull Request, Review, Merge ...

# === SYNCHRONIZACJA Z ZESPOŁEM ===

# 13. Pobranie najnowszych zmian od zespołu
git checkout main
git pull origin main

# 14. Aktualizacja swojego feature brancha
```

```
git checkout feature-my-experiment
git merge main  # lub git rebase main
```

---

8. 🧪 Zadania do samodzielnej pracy

✏️ Zadania podstawowe (1-8)

1. ✏️ **Zadanie 1 – Inicjalizacja pierwszego repozytorium**

Utwórz lokalne repozytorium Git dla projektu Data Science.

Wymagania:

- Utwórz folder `my_first_ds_project`
- Zainicjalizuj repozytorium Git (`git init`)
- Utwórz plik `README.md` z opisem projektu
- Dodaj plik do staging area i commituj
- Sprawdź status i historię (`git status` , `git log`)

Oczekiwany rezultat:

- Działające lokalne repozytorium Git
- Jeden commit w historii

(proste)

1. ✏️ **Zadanie 2 – Tworzenie .gitignore**

Utwórz plik `.gitignore` dla typowego projektu Data Science.

Wymagania:

- Dodaj reguły wykluczające: pliki `.csv` , foldery `data/` , `models/` , `__pycache__/` , `.ipynb_checkpoints/`
- Commituj `.gitignore`
- Sprawdź, że Git ignoruje pliki zgodnie z regułami

Oczekiwany rezultat:

- Działający `.gitignore`
- Test: utwórz plik `test.csv` - nie powinien być tracked przez Git

(proste)

1. ✏️ **Zadanie 3 – Historia zmian**

Wykonaj serię commitów i przeanalizuj historię.

Wymagania:

- Utwórz 5 różnych commitów (np. dodanie plików, edycje README)
- Użyj `git log --oneline` do zobaczenia historii
- Użyj `git log --graph --all` do zobaczenia graficznej reprezentacji
- Użyj `git show <commit-hash>` do zobaczenia szczegółów commita

Oczekiwany rezultat:

- 5 commitów w historii
- Zrozumienie, jak przeglądać historię zmian

(proste)

1. ✏️ **Zadanie 4 – Tworzenie branchy**

Utwórz kilka branchy i przełączaj się między nimi.

Wymagania:

- Utwórz branch `feature-data-loading`
- Utwórz branch `feature-preprocessing`
- Przełącz się na `feature-data-loading`
- Dodaj plik `load_data.py`, commituj
- Przełącz się na `master` i sprawdź, że pliku tam nie ma
- Sprawdź listę wszystkich branchy (`git branch`)

Oczekiwany rezultat:

- Co najmniej 2 branche
- Różne pliki na różnych branchach

(proste)

1. ✏️ **Zadanie 5 – Merge branchy**

Połącz dwa branche używając merge.

Wymagania:

- Mając branch `feature-data-loading` z poprzedniego zadania
- Przełącz się na `master`
- Merge branch `feature-data-loading` do mastera
- Sprawdź, że plik `load_data.py` jest teraz na masterze
- Usuń branch `feature-data-loading`

Oczekiwany rezultat:

- Plik z feature brancha dostępny na masterze
- Branch feature usunięty

(proste)

1. ✏️ **Zadanie 6 – Konto GitHub i pierwsze repo**

Załóż konto na GitHub i utwórz pierwsze zdalne repozytorium.

Wymagania:

- Załóż konto na github.com (jeśli nie masz)
- Utwórz nowe repozytorium na GitHub o nazwie `ds-learning`
- Dodaj opis i README przez interfejs GitHub
- Sklonuj repozytorium na lokalny komputer
- Dodaj nowy plik `notes.md`, commituj i wypchnij na GitHub

Oczekiwany rezultat:

- Publiczne repozytorium na GitHub
- Zmiany widoczne na GitHub po push

(proste)

1. ✏️ **Zadanie 7 – Push i Pull**

Zsynchronizuj lokalne repozytorium z GitHub.

Wymagania:

- Mając lokalne repo z Zadania 1
- Połącz je ze zdalnym repozytorium na GitHub
- Wypchnij wszystkie commity (`git push`)
- Zmień coś przez interfejs GitHub (edytuj README online)
- Pobierz zmiany lokalnie (`git pull`)

Oczekiwany rezultat:

- Lokalne i zdalne repo zsynchronizowane
- Zmiany z GitHub widoczne lokalnie

(proste)

1. ✏️ **Zadanie 8 – Diff i porównanie zmian**

Naucz się porównywać zmiany między wersjami.

Wymagania:

- Edytuj plik `README.md` (dodaj kilka linii)
- Przed commitowaniem użyj `git diff` do zobaczenia zmian
- Commituj zmiany
- Użyj `git diff HEAD~1 HEAD` do porównania ostatniego commita z poprzednim
- Użyj `git diff <commit-hash1> <commit-hash2>` do porównania dowolnych commitów

Oczekiwany rezultat:

- Zrozumienie, jak przeglądać różnice między wersjami

(proste)

✏️ Zadania średnie (9-12)

1. ✏️ **Zadanie 9 – Git Flow: Feature Branch**

Zasymuluj pełny Git Flow dla nowej funkcjonalności.

Dataset: Projekt analizy danych klientów (symulacja)

Wymagania:

- Utwórz branch `develop`
- Z develop utwórz `feature-customer-segmentation`
- Dodaj notebook `customer_segmentation.ipynb`
- Commituj zmiany
- Merge do `develop`
- Merge `develop` do `main`
- Otaguj release: `git tag -a v1.0 -m "Release 1.0"`

Oczekiwany rezultat:

- Kompletny Git Flow workflow
- Tagged release

(średnie)

1. ✏️ **Zadanie 10 – Rozwiązywanie konfliktów**

Celowo utwórz konflikt i rozwiąż go.

Wymagania:

- Utwórz branch `branch-a` i `branch-b` z tego samego punktu
- Na obu edytuj tę samą linię w tym samym pliku (różnie)
- Commituj na obu branchach
- Merge `branch-a` do `master`
- Spróbuj merge `branch-b` do `master` - pojawi się konflikt
- Ręcznie rozwiąż konflikt, edytując plik
- Commituj rozwiązanie konfliktu

Oczekiwany rezultat:

- Zrozumienie, jak rozwiązywać konflikty merge

(średnie)

1. ✏️ **Zadanie 11 – Praca z remote branches**

Współpraca z wieloma remote branches.

Wymagania:

- Utwórz branch lokalnie: `feature-model-training`
- Wypchnij na GitHub: `git push -u origin feature-model-training`
- Sklonuj to samo repo w innym folderze (symulacja kolegi)
- W drugim folderze edytuj ten sam branch, commituj, pushuj
- W pierwszym folderze pobierz zmiany: `git pull origin feature-model-training`
- Upewnij się, że zmiany są zsynchronizowane

Oczekiwany rezultat:

- Synchronizacja zmian między "dwoma osobami"

(średnie)

1. ✏️ **Zadanie 12 – Pull Request przez GitHub**

Utwórz Pull Request i wykonaj code review.

Wymagania:

- Utwórz nowy branch `feature-visualization`
- Dodaj plik z kodem (np. `visualize.py` z funkcją do wykresów)
- Pushuj branch na GitHub
- Utwórz Pull Request przez GitHub interface
- Napisz szczegółowy opis PR (co zmienia, dlaczego)
- Dodaj komentarz do PR (review samemu siebie lub poproś znajomego)
- Merge PR przez GitHub

Oczekiwany rezultat:

- Zamknięty i zmergowany Pull Request

(średnie)

🧠 Zadania wyzwanie (13-20)

1. 🧠 **Zadanie 13 – Kompletny projekt ML z Git Flow**

Stwórz pełny projekt ML od zera z profesjonalnym Git Flow.

Dataset: Titanic lub Iris

Wymagania:

- Struktura: `data/`, `notebooks/`, `src/`, `models/`
- `.gitignore` wykluczający dane i modele
- Branch `develop`
- Feature branch dla EDA: `feature-eda`
- Feature branch dla modelu: `feature-baseline-model`
- Każdy branch z minimum 3 commitami
- Pull Requests dla każdego brancha
- Release tag `v1.0` na main

Oczekiwany rezultat:

- Profesjonalnie zorganizowane repozytorium na GitHub
- Historia commitów pokazująca rozwój projektu

(challenge)

1. 🧠 **Zadanie 14 – Squash commits i clean history**

Naucz się "sprzątać" historię commitów.

Wymagania:

- Utwórz branch z 10 małymi commitami (symulacja chaotycznej pracy)
- Użyj `git rebase -i HEAD~10` do squashowania commitów
- Połącz wszystkie 10 commitów w 2-3 logiczne commity
- Przepisz commit messages na bardziej opisowe
- Pushuj "czystą" historię

Oczekiwany rezultat:

- Czytelna historia commitów zamiast chaosu

(challenge)

1. 🧠 **Zadanie 15 – Automatyzacja z GitHub Actions**

Utwórz GitHub Action, który automatycznie testuje kod.

Wymagania:

- Dodaj testy jednostkowe do projektu (np. `test_preprocessing.py` z pytest)
- Utwórz `.github/workflows/test.yml`
- Skonfiguruj workflow, który uruchamia testy przy każdym push
- Pushuj i sprawdź, czy testy przechodzą w GitHub Actions
- Dodaj badge "Tests Passing" do README

Oczekiwany rezultat:

- Automatyczne testy przy każdym push
- Badge w README

(challenge)

1. 🧠 **Zadanie 16 – Monorepo dla wielu projektów DS**

Zarządzaj wieloma projektami w jednym repo.

Wymagania:

- Struktura: `projects/project1/`, `projects/project2/`, `shared/`
- Każdy projekt ma własny `README.md` i structure
- Folder `shared/` zawiera wspólne utility functions
- Commituj zmiany logicznie (nie mieszaj projektów w jednym commicie)
- Używaj konwencji commitów: `[project1] opis` lub `[shared] opis`

Oczekiwany rezultat:

- Monorepo z wieloma projektami DS
- Czytelna historia commitów

(challenge)

1. 🧠 **Zadanie 17 – Git Hooks dla jakości kodu**

Dodaj pre-commit hook sprawdzający jakość kodu.

Wymagania:

- Zainstaluj `pre-commit` framework: `pip install pre-commit`
- Utwórz `.pre-commit-config.yaml`
- Dodaj hooki: black (formatowanie), flake8 (linting), trailing-whitespace

- Zainstaluj hooki: `pre-commit install`
- Spróbuj commitować kod z błędami - hook powinien zablokować
- Popraw kod i commituj ponownie

Oczekiwany rezultat:

- Automatyczne sprawdzanie jakości kodu przed każdym commitem

(challenge)

1. 🧠 **Zadanie 18 – Zaawansowane Git: Cherry-pick i Stash**

Użyj zaawansowanych funkcji Git.

Wymagania:

- **Cherry-pick**: Weź jeden commit z brancha A i aplikuj na branchu B
- **Stash**: Rozpocznij pracę na branchu, przełącz się na inny nie commitując (użyj `git stash`), wróć i odzyskaj zmiany (`git stash pop`)
- Wykonaj obie operacje i zadokumentuj kroki

Oczekiwany rezultat:

- Zrozumienie cherry-pick i stash

(challenge)

1. 🧠 **Zadanie 19 – GitHub Project Board dla projektu ML**

Zarządzaj projektem ML używając GitHub Projects.

Wymagania:

- Utwórz GitHub Project Board (Kanban)
- Kolumny: Backlog, To Do, In Progress, Review, Done
- Dodaj Issues dla każdego etapu projektu ML: data collection, EDA, preprocessing, modeling, evaluation, deployment
- Linkuj Issues do Pull Requests
- Przenoś Issues między kolumnami w miarę postępu

Oczekiwany rezultat:

- Kompletny Project Board pokazujący workflow projektu ML

(challenge)

1. 🧠 **Zadanie 20 – DVC dla wersjonowania danych i modeli**

Użyj DVC (Data Version Control) zamiast Git dla dużych plików.

Dataset: Dowolny dataset > 100 MB

Wymagania:

- Zainstaluj DVC: `pip install dvc`
- Inicjalizuj DVC w repo: `dvc init`
- Dodaj duży plik danych: `dvc add data/large_dataset.csv`
- Commituj plik `.dvc` do Git (nie sam dataset)
- Skonfiguruj remote storage (np. DVC remote lub symulacja lokalnie)
- `dvc push` i `dvc pull`
- Zmień dane, utwórz nową wersję, commituj

Oczekiwany rezultat:

- Wersjonowanie dużych plików z DVC
- Git repo pozostaje lekkie

(challenge)

---

9. 📚 Podsumowanie

W tej lekcji poznałeś fundamentalne narzędzie każdego Data Scientist - **Git i GitHub**. Oto kluczowe wnioski:

Czego się nauczyłeś:

1. **Kontrola wersji jest niezbędna** w Data Science:

- Historia zmian w kodzie i notebookach
- Bezpieczeństwo i możliwość powrotu do poprzednich wersji
- Podstawa współpracy w zespole

1. **Git - podstawowe operacje:**

- `git init` - inicjalizacja repozytorium
- `git add`, `git commit` - zapisywanie zmian
- `git status`, `git log` - kontrola stanu i historii
- `git diff` - porównywanie zmian

1. **Branching i Git Flow:**

- Branche pozwalają na równoległą pracę nad różnymi funkcjonalnościami
- Git Flow definiuje strategię zarządzania branchami (main, develop, feature, release, hotfix)
- `git merge` łączy zmiany z różnych branchy

1. **GitHub - współpraca i sharing:**

- Centralne repozytorium dla zespołu
- Pull Requests jako mechanizm code review
- Issues do zarządzania zadaniami i bugami
- GitHub Actions do automatyzacji (CI/CD)

1. **Aspekt środowiskowy:**

- Git redukuje duplikację plików (oszczędność storage i energii)
- `.gitignore` wykluczający duże pliki
- Repozytoria chmurowe zamiast lokalnych kopii
- Efektywna współpraca zmniejszająca transfer danych

Jak to się łączy z poprzednimi lekcjami:

- **Lekcja 1 (Wprowadzenie):** Poznałeś czym jest Data Science - Git to narzędzie, które umożliwia profesjonalną pracę w tej dziedzinie
- **Lekcja 2 (Środowisko pracy):** Skonfigurowałeś środowisko (Colab, Jupyter) - teraz wiesz, jak wersjonować swoje notebooki i projekty

Przygotowanie do następnych tematów:

W kolejnych lekcjach będziesz:

- Uczyć się **Pythona dla Data Science** - Git pozwoli Ci śledzić ewolucję Twojego kodu

- Pracować z **danymi i bibliotekami** (Pandas, NumPy) - będziesz commitować skrypty przetwarzania danych
- Budować **modele ML** - Git pomoże Ci eksperymentować z różnymi podejściami na oddzielnych branchach
- Wdrażać **projekty do produkcji** - GitHub Actions zautomatyzuje deployment

Git jest fundamentem profesjonalnego Data Science! Od teraz każdy projekt twórz z repozytorium Git, commituj zmiany regularnie i dziel się kodem przez GitHub. To nie tylko dobre praktyki, ale też sposób na budowanie portfolio, które przyciągnie uwagę rekruterów.

Refleksja nad aspektem środowiskowym:

Używając Git i GitHub:

- **Oszczędzasz ~90% przestrzeni dyskowej** (vs wielokrotne kopie plików)
- **Redukujesz transfer danych** w zespole (centralne repo vs wysyłanie emaili)
- **Minimalizujesz zużycie energii** serwerów (efektywne przechowywanie)
- **Promujesz open-source** - dzielenie się kodem pozwala innym unikać "wynajdywania koła na nowo", co oszczędza energię i zasoby całej społeczności

Zrównoważony Data Science to także efektywne zarządzanie kodem i danymi!

---

10. 📖 Dodatkowe Zasoby

Oficjalna dokumentacja:

- Git Documentation - pełna dokumentacja Git
- GitHub Docs - przewodniki GitHub
- Pro Git Book - darmowa książka online (po polsku też dostępna)

Interaktywne tutoriale:

- Learn Git Branching - wizualna, interaktywna nauka Git (po polsku!)
- GitHub Learning Lab - kursy GitHub od podstaw

Narzędzia:

- GitKraken - graficzny interfejs dla Git
- nbdime - diffowanie i mergowanie Jupyter Notebooks
- DVC - wersjonowanie danych i modeli ML
- pre-commit - framework do Git hooks

Cheat sheets:

- Git Cheat Sheet (PDF)
- GitHub Flow Guide

Datasety do praktyki:

- Kaggle - tysiące datasetów do eksperymentów
- UCI ML Repository - klasyczne datasety ML
- GitHub Awesome Datasets - lista publicznie dostępnych datasetów

Best practices dla Data Science:

- Cookiecutter Data Science - template struktury projektu DS
- Good practices for DS on GitHub

Powodzenia w Twojej przygodzie z Git i GitHub! 🚀

ROZPOCZNIJ KARIERĘ W DATA SCIENTIST

Gotowy na następny krok?

Ten materiał to tylko początek. Dołącz do kursu Data Scientist Online + AI i zdobądź praktyczne umiejętności pod okiem ekspertów.

- 5,5 miesiąca intensywnej nauki
- Zajęcia na żywo 2x w tygodniu
- Certyfikat + pomoc w znalezieniu pracy

Zobacz szczegóły kursu →

Zadzwoń do nas

info@learnit.com.pl +48 573-580-990 learnit.com.pl